BACK MATTER

Glossary

Most of these terms are defined on first use in the body of the book. This page collects those definitions in one place, in the sense the book gives them rather than any wider dictionary sense.

TERM	DEFINITION
Specification	A precise, technology-independent description of what a system must do: which actors are involved, what the rules are, and what counts as success or failure. Chapters 2 to 5 assemble its core from vision, requirements, use cases, and a domain model. Chapters 6 and 7 add the architecture constraints and skills an implementation also needs, so from that point the word covers all six.
Vision	The explicit statement of why a system exists and for whom, written in business language before any requirement is captured. It is stable over time and constrains scope before any architecture decision locks anything in.
Requirement	An observable, testable statement about system behavior that stays silent on implementation. A requirement states a rule; it does not say how the system enforces it.
Use case	A complete description of one interaction: an actor, preconditions, a main scenario, alternative flows, postconditions, and the business rules that govern it. It is the unit AI implements best, because it specifies a whole interaction rather than a single property.

TERM	DEFINITION
Domain model	A description of the business objects a system runs on, and the relationships and constraints between them, independent of how anything is stored. It captures business meaning, not database design.
Entity	A thing the business cares about that has its own identity and attributes that can change over time, such as a patient or an appointment. Two entities with identical attributes are still different things if their identities differ.
Value object	An immutable concept with no identity of its own, meaningful only in the context of the entity that holds it, such as an address or a time slot. Value objects are typically attributes of entities rather than entities themselves.
Enumeration	An attribute limited to a fixed, named set of valid values, such as an appointment status of Scheduled, Completed, Cancelled, or NoShow. It is neither an entity nor a value object, and it needs revisiting whenever the business adds a case the existing values do not cover.
Architecture	The set of decisions that apply to every feature in a codebase: layering, dependency direction, naming, module boundaries, and how components communicate. Documented once, it replaces re-explaining the same structural rules in every request made to AI.
Skill	Reusable implementation knowledge for a specific technology stack: how an organization writes tests, handles errors, or structures a service. A skill is distinct from architecture and stays a versioned, living document as conventions change.
Context	The information an AI agent already has before being asked to do something, as opposed to what a single prompt spells out at the moment of the request. This book's reading is that complete, relevant context does more for the result than careful wording does.
Protocol	A specification of what a system does, kept independent of how any one implementation builds it, the sense this book borrows from internet RFCs. A business process specified as a protocol can be implemented differently by different teams and still interoperate.
Implementation	The technology-specific realization of a specification: the working code, database schema, and configuration that make a specified behavior run. Implementation translates a decision already made; it does not make the decision.

Notes and References

The claims in this book carry different kinds of standing. The table below sorts the sources behind them by that standing: reported practice from one methodology, published technical standards, and established professional practice this book draws on rather than originates. Where a later RFC has superseded one of these standards, the row names both. Appendix B reads the originals as specifications that outlived their implementations, so the original numbers still matter.

CLAIM TYPE	SOURCE
Reported practice	Simon Martinelli's specification-driven development methodology, which he calls the AI Unified Process, credited throughout this book as the source of its core recommendations: `unifiedprocess.ai`
Reported practice	Simon Martinelli, "Lessons from Spec-driven Development," AI Native DevCon, June 2026: `youtu.be/odbNXv9xXjc`
Reported practice	The AI Unified Process packaged as Claude Code plugins, Apache 2.0: `github.com/AI-Unified-Process/marketplace`
Standard	RFC 791, Internet Protocol
Standard	RFC 793, Transmission Control Protocol; obsoleted in 2022 by RFC 9293, Transmission Control Protocol (TCP)
Standard	RFC 7230, HTTP/1.1: Message Syntax and Routing; obsoleted in 2022 by RFC 9112, HTTP/1.1, and RFC 9110, HTTP Semantics
Standard	RFC 5322, Internet Message Format
Standard	RFC 3339, Date and Time on the Internet: Timestamps
Standard	RFC 7807, Problem Details for HTTP APIs; obsoleted in 2023 by RFC 9457, Problem Details for HTTP APIs
Standard	ISO 8601, date and time representation
Standard	ITU-T E.164, the international public telecommunication numbering plan

CLAIM TYPE	SOURCE
Established practice	Eric Evans, domain-driven design
Established practice	The Agile Manifesto
Established practice	Architecture decision records
Established practice	arc42
Established practice	The C4 model
Established practice	The Model Context Protocol
Established practice	Martin Fowler's Strangler Fig pattern

NOTE

The case studies in Appendix A are presented as reported by the organizations involved, with names and identifying details changed. This book does not carry forward the aggregate percentage-range statistics that appeared in the original source manuscript's packaging material, because those ranges do not trace back to any individual case's figures. Where such a number appears in this book, it is attributed to the single case that reported it.

Index

Page numbers are assigned after final pagination and are not yet included. Entries below are grouped by the four categories this book uses, not alphabetized within them.

Concepts

Artifacts and Templates

Examples

Risks and Failure Modes